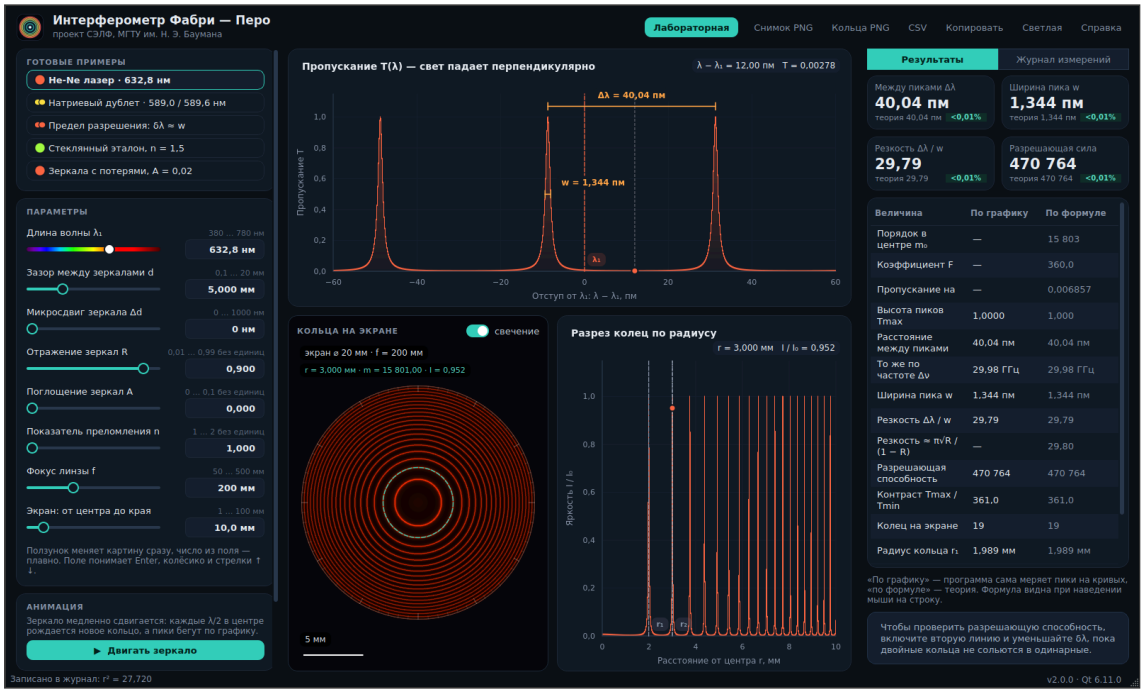
Московский государственный технический университет
им. Н. Э. Баумана

Проект СЭЛФ

МОДЕЛЬ ИНТЕРФЕРОМЕТРА ФАБРИ — ПЕРО

ЧАСТЬ 2. ВИЗУАЛЬНАЯ ЧАСТЬ ПРОГРАММЫ

Методическое пособие по коду программы



Разобрано, как устроено окно программы: библиотека Qt, раскладка, тёмная и светлая темы, цвет света, связка «ползунок + поле ввода», плавные переходы, рисование графиков и картины колец, измерения курсором, сохранение результатов, сборка программы для Windows 7–11, macOS и Linux.

Расчётная часть (формулы, измерения, проверки) разобрана в первой части пособия.

Москва, 2026

1. QT ЗА ДЕСЯТЬ МИНУТ

Окно программы построено на библиотеке **Qt** (в Python — пакет PyQt). Нужно знать пять понятий.

Виджет — любой элемент окна: кнопка `QPushButton`, надпись `QLabel`, ползунок `QSlider`, поле ввода числа `QDoubleSpinBox`. Своё окно, график или картина колец — тоже виджеты: классы, унаследованные от `QWidget`.

Компоновка (layout) расставляет виджеты: `QVBoxLayout` — столбиком, `QHBoxLayout` — в строку, `QGridLayout` — таблицей, `QSplitter` — с перетаскиваемыми границами. Размеры считаются сами при изменении окна.

Сигналы и слоты. Виджет сообщает о событии сигналом, к которому подключается функция-слот:

```
button.clicked.connect(self.save_png)            # щелчок → сохранить картинку
self.slider.valueChanged.connect(self.slider_moved)  # ползунок сдвинули → обновить поле
```

События. Qt сам вызывает методы виджета: `paintEvent` — «нарисуй себя», `mouseMoveEvent` — мышь сдвинулась, `wheelEvent` — колёсико, `resizeEvent` — изменился размер. Чтобы перерисовать виджет, вызывают `self.update()`: Qt вызовет `paintEvent`, когда будет удобно.

Таймер `QTimer` вызывает функцию через заданное время или регулярно — на нём построены плавные переходы и анимация зеркала.

Рисует виджет объект `QPainter`: линии, текст, скруглённые прямоугольники, готовые картинки `QImage`. Графики и кольца в программе нарисованы им, без сторонних библиотек вроде matplotlib: так программа легче, быстрее и работает на Windows 7.

1.1. PyQt6 или PyQt5

Qt 6 не работает на Windows 7, 8 и 8.1, поэтому для них программа собирается с PyQt5. Разница между ними собрана в одном маленьком модуле `qt.py`; остальной код одинаков для обеих версий.

fabry_perot/qt.py — qt

```
import os

QT6 = False
if os.environ.get("FABRY_QT") != "5":
    try:
        from PyQt6 import QtCore, QtGui, QtWidgets
        QT6 = True
    except ImportError:
        pass
if not QT6:
    from PyQt5 import QtCore, QtGui, QtWidgets

Qt = QtCore.Qt
Signal = QtCore.pyqtSignal
QPointF, QRectF, QLineF, QSize = QtCore.QPointF, QtCore.QRectF, QtCore.QLineF, QtCore.QSize
QBrush, QColor, QFont, QIcon, QImage = QtGui.QBrush, QtGui.QColor, QtGui.QFont, QtGui.QIcon,
QtGui.QImage
QLinearGradient, QRadialGradient = QtGui.QLinearGradient, QtGui.QRadialGradient
```

```
QPainter, QPalette, QPen, QPixmap, QPolygonF = (QtGui.QPainter, QtGui.QPalette, QtGui.QPen,
                                                QtGui.QPixmap, QtGui.QPolygonF)
QAction = QtGui.QAction if QT6 else QtWidgets.QAction


def event_pos(event):
    return event.position() if QT6 else event.localPos()
```

Приём `try … except ImportError`: пробуем подключить PyQt6, а если его нет — PyQt5. Всё различие — в двух мелочах: где лежит класс `QAction` и как называется метод «положение мыши».

2. ОКНО И РАСКЛАДКА

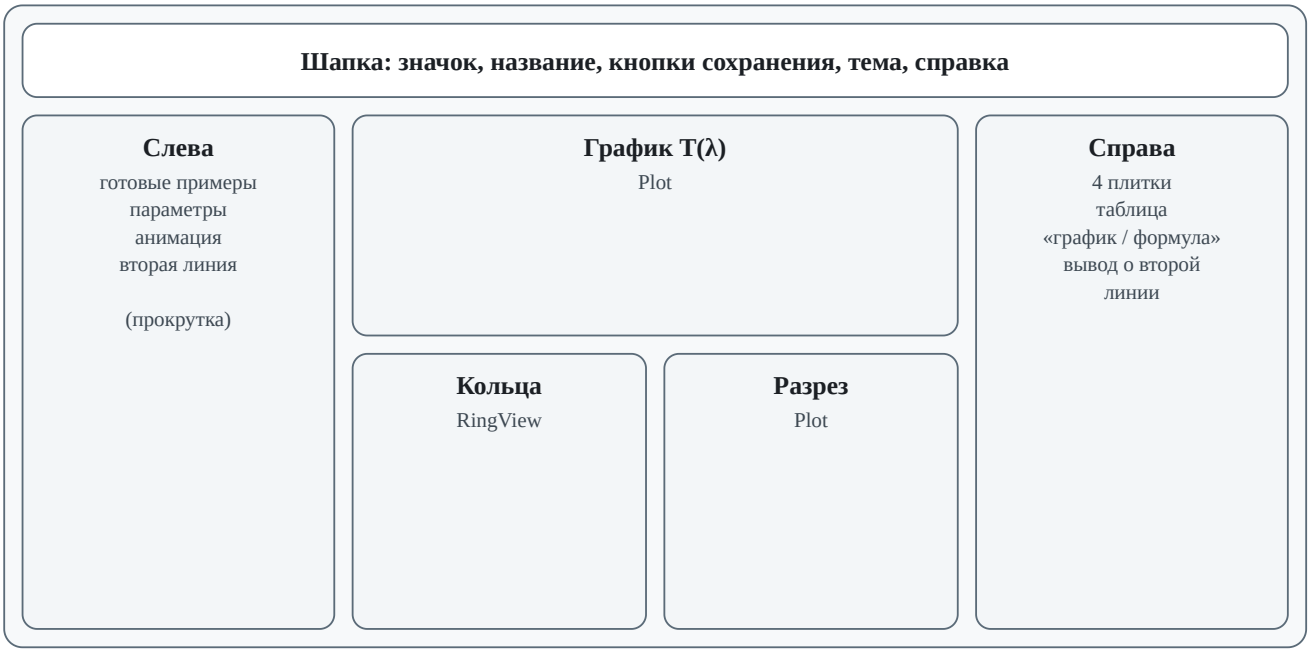


Рис. 1. Раскладка окна

Окно (класс `MainWindow` в `window.py`) строится четырьмя методами: `build_header` (шапка), `build_controls` (левая колонка), `build_views` (графики и кольца), `build_results` (правая колонка). Три колонки стоят в разделителе `QSplitter`; при растяжении окна растут графики (`setStretchFactor(1, 1)`). Левая колонка лежит в `QScrollArea` — на маленьком экране её можно прокрутить. Минимальная ширина окна около 1320 точек, поэтому всё помещается на распространённых в аудиториях мониторах 1366×768.

fabry_perot/window.py — MainWindow.build_views

```
def build_views(self):
    self.plot_t = Plot("Пропускание T(λ) — свет падает перпендикулярно", "λ − λ₁", "Пропускание
T")
    self.plot_r = Plot("Разрез колец по радиусу", "r", "Яркость I / I₀")
    self.ring_view = RingView()
    self.ring_view.hovered.connect(self.plot_r.set_hover)
    self.plot_r.hovered.connect(self.ring_view.set_hover)
    self.plot_t.picked.connect(lambda x, y: self.pick(lambda: self.journal.plot_pick(x,
self.plot_t.xunit, y)))
    self.ring_view.picked.connect(lambda r, y: self.pick(lambda: self.journal.ring_pick(r, y)))
    self.plot_r.picked.connect(lambda r, y: self.pick(lambda: self.journal.ring_pick(r, y)))

    screen, layout = make_card(None, "screen")
```

```
    layout.setContentsMargins(12, 10, 12, 12)
    top = QtWidgets.QHBoxLayout()
    top.addWidget(card_title("Кольца на экране"))
    top.addStretch()
    self.glow_switch = ToggleSwitch("свечение", "#C9D3DF")
    self.glow_switch.setChecked(True)
    self.glow_switch.setToolTip("Мягкий ореол вокруг ярких колец, как на фотографии")
    self.glow_switch.toggled.connect(self.ring_view.set_glow)
    top.addWidget(self.glow_switch)
    self.ring_view.setToolTip("Колёсико — лупа, перетаскивание — сдвиг, двойной щелчок — весь
экран, "
                              "правый щелчок — записать радиус в журнал")
    layout.addLayout(top)
    layout.addWidget(self.ring_view, 1)

    bottom = QtWidgets.QSplitter(Qt.Orientation.Horizontal)
    bottom.addWidget(screen)
    bottom.addWidget(self.plot_r)
    bottom.setSizes([470, 530])
    views = QtWidgets.QSplitter(Qt.Orientation.Vertical)
    views.addWidget(self.plot_t)
    views.addWidget(bottom)
    views.setSizes([420, 580])
    for splitter in (bottom, views):
        splitter.setChildrenCollapsible(False)
        splitter.setHandleWidth(12)
    return views
```

Обратите внимание на две строки с `hovered.connect`: наведение мыши на кольца показывает тот же радиус на разрезе, и наоборот. Виджеты ничего не знают друг о друге — их связывает окно через сигналы.

3. ОФОРМЛЕНИЕ

3.1. Палитры и таблица стилей

Все цвета собраны в двух словарях — тёмная и светлая тема (`theme.py`). Виджеты не хранят цветов, а берут их из общего объекта `THEME` в момент рисования. Поэтому смена темы — это смена словаря, новая таблица стилей и перерисовка окна.

fabry_perot/theme.py — DARK

```
DARK = {
    "name": "dark",
    "bg": "#0A0F16",
    "surface": "#111923",
    "raised": "#172230",
    "border": "#1F2B3A",
    "line": "#2A3A4E",
    "text": "#E7EDF4",
    "muted": "#8795A8",
    "faint": "#5B6A7D",
    "grid": "#18222F",
    "axis": "#2E3D50",
```

Внешний вид стандартных виджетов задаётся **таблицей стилей Qt** (QSS) — языком, похожим на CSS веб-страниц. Таблица собирается f-строкой, в которую подставлены цвета текущей темы:

fabry_perot/theme.py — stylesheet

```
def stylesheet():
    p = THEME.p
```

```
    return f"""
QWidget {{ color: {p['text']}; font-size: 13px; }}
QMainWindow, QScrollArea, QWidget#page, QWidget#side {{ background: {p['bg']}; }}
QLabel {{ background: transparent; }}
QLabel#appTitle {{ font-size: 17px; font-weight: 700; }}
QLabel#appSub, QLabel#hint {{ color: {p['muted']}; font-size: 12px; }}
QLabel#h2 {{ font-size: 16px; font-weight: 700; }}
QLabel#cardTitle {{ font-size: 11px; font-weight: 700; color: {p['muted']}; }}
QLabel#limits {{ color: {p['faint']}; font-size: 11px; }}
QLabel#tileCaption {{ color: {p['muted']}; font-size: 12px; }}
```

Селектор `QLabel#badge[kind="ok"]` выбирает надписи с именем `badge` и свойством `kind = "ok"`. Меняя свойство, программа перекрашивает значок расхождения (зелёный, жёлтый, красный) и подложку вывода о второй линии. После смены свойства стиль нужно применить заново — это делает функция `polish`:

fabry_perot/widgets.py — polish

```
def polish(widget, kind):
    widget.setProperty("kind", kind)
    widget.style().unpolish(widget)
    widget.style().polish(widget)
```

3.2. Цвет света

Кривые и кольца окрашены в цвет самого света. Цвет по длине волны вычисляется кусочно-линейной схемой (схема Брутона): от фиолетового через синий, голубой, зелёный, жёлтый к красному. У краёв видимого диапазона яркость снижается — глаз там чувствует свет хуже.

fabry_perot/theme.py — wavelength_rgb

```
def wavelength_rgb(nm):
    if nm < 440:
        r, g, b = (440 - nm) / 60, 0.0, 1.0
    elif nm < 490:
        r, g, b = 0.0, (nm - 440) / 50, 1.0
    elif nm < 510:
        r, g, b = 0.0, 1.0, (510 - nm) / 20
    elif nm < 580:
        r, g, b = (nm - 510) / 70, 1.0, 0.0
    elif nm < 645:
        r, g, b = 1.0, (645 - nm) / 65, 0.0
    else:
        r, g, b = 1.0, 0.0, 0.0
    if nm < 420:
        k = 0.3 + 0.7 * (nm - 380) / 40
    elif nm > 700:
        k = 0.3 + 0.7 * (780 - nm) / 80
    else:
        k = 1.0
    return r * k, g * k, b * k
```

На графике тот же цвет подгоняется под фон: на тёмном фоне тусклые фиолетовые и тёмно-красные цвета осветляются, на белом — яркие жёлтые затемняются. Для этого цвет переводится в систему HSL (оттенок, насыщенность, светлота), и меняется только светлота — оттенок сохраняется.

fabry_perot/theme.py — line_color

```
def line_color(nm):
    r, g, b = wavelength_rgb(nm)
```

```
    k = max(r, g, b) or 1.0
    c = QColor.fromRgbF(r / k, g / k, b / k)
    h, s, light = max(c.hslHueF(), 0.0), c.hslSaturationF(), c.lightnessF()
    if THEME.dark:
        light = max(light, 0.62)
    else:
        yellowish = max(0.0, 1 - abs(h - 0.17) / 0.12)
        light = min(light, 0.42 - 0.12 * yellowish)
    return QColor.fromHslF(h, min(s, 0.95), light).name()
```

4. ПАРАМЕТРЫ: ПОЛЗУНОК И ПОЛЕ ВВОДА

Все параметры описаны одной таблицей `PARAMS` в `params.py`: ключ, название, единица, пределы, значение по умолчанию, число знаков, шаг, подсказка, логарифмическая шкала. Строки на экране создаются по ней в цикле — чтобы добавить параметр, достаточно дописать строку таблицы.

fabry_perot/params.py — PARAMS

```
PARAMS = [
    ("lam", "Длина волны λ₁", "нм", 380.0, 780.0, 632.8, 1, 0.1,
     "Цвет света: 400 нм — фиолетовый, 530 нм — зелёный, 630 нм — красный.", False),
    ("d", "Зазор между зеркалами d", "мм", 0.1, 20.0, 5.0, 3, 0.01,
     "Расстояние между зеркалами. Больше d — пики ближе друг к другу, колец больше.", False),
    ("dd", "Микросдвиг зеркала Δd", "нм", 0.0, 1000.0, 0.0, 0, 1.0,
     "Добавка к зазору в нанометрах: полный зазор = d + Δd. Это тот же зазор, только тонкая шкала:
"
     "сдвиг на λ/2 рождает в центре одно новое кольцо.", False),
    ("R", "Отражение зеркал R", "", 0.01, 0.99, 0.9, 3, 0.01,
```

Каждая строка — объект `ParamRow`: ползунок и поле, связанные между собой. У ползунка 1000 положений, значение пересчитывается линейно, а для разности длин волн $\delta\lambda$ (0,01…2000 пм — пять порядков) — по логарифму, иначе малые значения нельзя было бы выставить:

fabry_perot/params.py — ParamRow.to_slider

```
def to_slider(self, value):
    if self.log:
        t = math.log(value / self.lo) / math.log(self.hi / self.lo)
    else:
        t = (value - self.lo) / (self.hi - self.lo)
    return round(t * self.STEPS)
```

fabry_perot/params.py — ParamRow.from_slider_pos

```
def from_slider_pos(self, pos):
    t = pos / self.STEPS
    return self.lo * (self.hi / self.lo) ** t if self.log else self.lo + t * (self.hi - self.lo)
```

Главное — окно должно знать, *откуда* пришло новое значение: от ползунка (картина меняется сразу) или из поля ввода (плавный переход). Для этого в строке есть флажок `from_slider`:

fabry_perot/params.py — ParamRow.slider_moved

```
def slider_moved(self, pos):
    self.from_slider = True
    self.spin.setValue(self.from_slider_pos(pos))
    self.from_slider = False
```

fabry_perot/params.py — ParamRow.spin_changed

```
def spin_changed(self, value):
    if not self.from_slider:
        self.show_on_slider(value)
    if self.from_slider:
        smooth = False
    elif self.forced is not None:
        smooth = self.forced
    elif self.spin.stepping:
        smooth = "fast"
    else:
        smooth = True
    self.on_change(self.key, smooth)
```

Строка `setKeyboardTracking(False)` в конструкторе поля важна: без неё при наборе числа «12» поле сообщило бы сначала «1», потом «12», и картина «поплыла» бы сначала к единице. С ней значение приходит только после Enter или ухода из поля.

5. ПЛАВНЫЕ ПЕРЕХОДЫ

Когда число вводится с клавиатуры или выбирается пример, картина не прыгает, а за 0,85 с «переплывает» к новому состоянию. Программа хранит два набора значений: в полях ввода — *цель*, в словаре `shown` — то, что сейчас на картинке. Переход — это постепенное изменение `shown` от старого значения к цели.

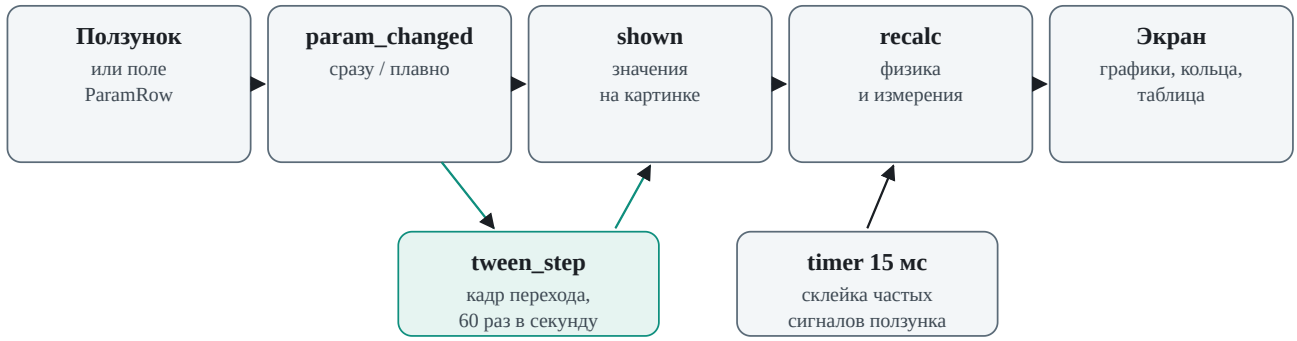


Рис. 2. Путь данных от ползунка до экрана

fabry_perot/window.py — MainWindow.param_changed

```
def param_changed(self, key, smooth):
    if key not in self.params:
        return
    target = self.params[key].value()
    if key == "dd" and self.scan_timer.isActive():
        self.scan_timer.stop()
        self.update_scan_button()
    if smooth and self.ready and key in self.shown:
        duration = FAST_MS if smooth == "fast" else TWEEN_MS
        self.tweens[key] = (self.shown[key], target, self.clock.elapsed(), self.params[key].log,
duration)
        if not self.tween_timer.isActive():
            self.tween_timer.start()
    else:
        self.tweens.pop(key, None)
        self.shown[key] = target
        self.schedule()
    if not self.applying:
        self.check_preset(None)
```

Переход начинается *с того, что сейчас на картинке* (`self.shown[key]`), а не со старого значения поля. Поэтому, если ввести новое число посреди перехода, картина плавно развернётся к новой цели без рывка.

fabry_perot/window.py — MainWindow.tween_step

```
def tween_step(self):
    now = self.clock.elapsed()
    for key, (a, b, start, log, duration) in list(self.tweens.items()):
        t = min((now - start) / duration, 1.0)
        e = ease(t)
        self.shown[key] = a * (b / a) ** e if log and a > 0 and b > 0 else a + (b - a) * e
        if t >= 1.0:
            self.shown[key] = b
            del self.tweens[key]
    if not self.tweens:
        self.tween_timer.stop()
    self.recalc()
```

Каждые 16 мс (около 60 раз в секунду) таймер вызывает `tween_step`. Доля пройденного пути t считается **по часам** (`QElapsedTimer`), а не по числу кадров: на медленном компьютере кадров будет меньше, но переход всё равно займёт 0,85 с. Чтобы движение было естественным — с разгоном и торможением, — доля t пропускается через функцию плавности (рис. 3):

$$e(t) = \begin{cases} 4t^3, & t < 0,5, \\ 1 - \dfrac{(2-2t)^3}{2}, & t \geq 0,5\,. \end{cases} \tag{1}$$

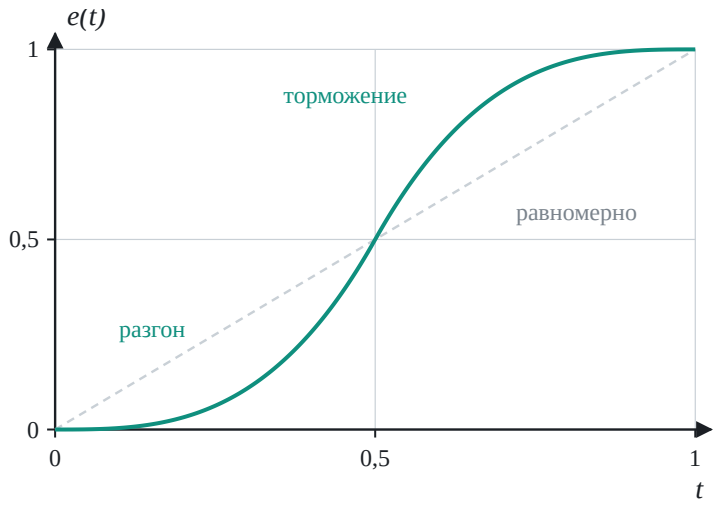


Рис. 3. Функция плавности: медленный старт, быстрая середина, мягкая остановка

fabry_perot/window.py — ease

```
def ease(t):
    return 4 * t ** 3 if t < 0.5 else 1 - (2 - 2 * t) ** 3 / 2
```

Для $\delta\lambda$ переход идёт по логарифму: $a\,(b\,/\,a)^{e}$ вместо $a + (b - a)\,e$. Иначе при переходе от 1 пм к 1000 пм картина почти всё время показывала бы большие значения.

Анимация «Двигать зеркало» устроена проще: таймер каждые 30 мс прибавляет к сдвигу зеркала $\lambda\,/\,(2n)\,/\,56$ — одно новое кольцо примерно за 1,7 с.

fabry_perot/window.py — MainWindow.scan_step

```
def scan_step(self):
    lam, n = self.shown["lam"], self.shown["n"]
    end = seamless_shift(lam, n, self.params["dd"].hi)
    value = self.shown["dd"] + lam / (2 * n) / 56
    if value > end:
        value -= end
    self.shown["dd"] = value
    self.params["dd"].display(value)
    self.recalc()
```

6. ПЕРЕСЧЁТ

Метод `recalc` — сердце окна. Он берёт значения из `shown`, переводит их в СИ, считает характеристики по формулам, выполняет измерения и передаёт данные графикам, картине колец и таблице. Частые сигналы ползунка склеиваются одноразовым таймером на 15 мс: пересчёт выполняется не на каждое движение, а не чаще 60 раз в секунду.

fabry_perot/window.py — MainWindow.recalc

```
def recalc(self):
    self.timer.stop()
    v = self.effective()
    lam = v["lam"] * 1e-9
    d = v["d"] * 1e-3 + v["dd"] * 1e-9
    R, n = v["R"], v["n"]
    A = min(v["A"], 1 - R)
    f, half = v["f"] * 1e-3, v["screen"] * 1e-3
    L = v["L"] * 1e-3 if self.lab is not None else f
    rho = defocus_radius(L, f)
    two = self.second.isChecked()
    lam2 = lam + v["dlam"] * 1e-12
    th = theory(lam, d, n, R, f, A)
    F, fsr, tmax = th["F"], th["fsr"], th["tmax"]
    hidden = self.lab is not None and self.unknown_doublet.isChecked()
    if not two:
        self.lam2_label.setText("Вторая линия выключена")
    elif self.lab is not None:
        self.lam2_label.setText("λ₂ = λ₁ + скрытая разность" if hidden else f"λ₂ = λ₁ +
{num(v['dlam'], 5)} пм")
    else:
        self.lam2_label.setText(f"λ₂ = {num(lam2 * 1e9, 9)} нм")

    x = lam + np.linspace(-1.5 * fsr, 1.5 * fsr, curve_samples(3 * fsr, th["width"], fsr))
    T = airy(phase(x, d, n), F, tmax)
    meas = measure_curve(x, T, lam)
    self.curve = (x, T)
    lo, hi = -1.5 * fsr, (lam2 - lam if two else 0.0) + 1.5 * fsr
    self.show_transmission(lam, lam2 if two else None, lo, hi, meas, d, n, F, tmax, hidden)

    lines = [(lam, "λ₁")] + ([(lam2, "λ₂")] if two else [])
    caption = f"экран ⌀ {short(round(2 * v['screen'], 1))} мм · f = {short(round(v['f']))} мм"
    if self.lab is not None:
        caption += f" · L = {short(round(v['L'], 1))} мм"
    self.ring_view.show_order = self.lab is None
    self.ring_view.show_data([(wl, wavelength_rgb(wl * 1e9)) for wl, _ in lines], half, caption,
                             (d, n, L), F, tmax, rho)

    measured = predicted = ring_radii(lam, d, n, L, half)
    self.radii = (measured, predicted)
    marks = [] if self.lab is not None else [
        (rk * 1e3, THEME["muted"], label) for rk, label in zip(measured[:2], ("r₁", "r₂"))]
    share = len(lines)

    def sin_out(x_mm):
        r = x_mm * 1e-3
```

```
        return r / np.hypot(r, L)

    def ring_curve(wl, name):
        if rho > 0:
            def value(x_mm):
                r = np.asarray(x_mm, dtype=float) * 1e-3
                return screen_mean(np.maximum(r - rho, 0.0), r + rho, wl, d, n, L, F, tmax) /
share
            return Curve(value, sampled_span(value), line_color(wl * 1e9), name if two else "I /
I₀")

        def value(x_mm):
            return airy(phase(wl, d, n, sin_out(x_mm)), F, tmax) / share

        def span(edges):
            lo_, hi_ = airy_range(phase(wl, d, n, sin_out(edges)), F, tmax)
            return lo_ / share, hi_ / share
        return Curve(value, span, line_color(wl * 1e9), name if two else "I / I₀")

    curves = [ring_curve(wl, name) for wl, name in lines]
    if two:
        curves.append(self.sum_curve(list(curves)))
    self.plot_r.show_data("Расстояние от центра r, мм", "мм", (0.0, v["screen"]), (0.0, 1.15),
                          curves, marks)

    self.fill_table(lam, th, meas, measured, predicted)
    kind, text = self.verdict_text(two, lam2 - lam, th)
    self.verdict.setText(text)
    if self.verdict.property("kind") != kind:
        polish(self.verdict, kind)
```

Графику передаются не массивы точек, а **формулы** — объекты `Curve` с двумя функциями: значение в точке (для курсора) и наименьшее/наибольшее значение в каждом столбце пикселей (для рисования). Функции создаются прямо внутри `recalc` и «помнят» текущие d, n, F (это называется *замыкание*).

fabry_perot/widgets.py — Curve

```
class Curve:
    def __init__(self, value, span, color, name):
        self.value, self.span, self.color, self.name = value, span, color, name

    def reading(self, x):
        return float(self.value(np.array([x]))[0])
```

7. ГРАФИК

7.1. Рисунок-«кэш» и перекрестие

Класс `Plot` рисует сетку, подписи, кривые, метки линий и отрезки измерений. Всё, кроме курсора, рисуется один раз в картинку-«кэш» `self.base`. При движении мыши на экран копируется готовая картинка и поверх — перекрестие со значениями. Поэтому курсор движется плавно даже на слабом компьютере.

fabry_perot/widgets.py — Plot.paintEvent

```
def paintEvent(self, event):
    ratio = self.devicePixelRatioF()
    if self.base is None:
        self.base = QPixmap(int(self.width() * ratio), int(self.height() * ratio))
        self.base.setDevicePixelRatio(ratio)
        self.base.fill(Qt.GlobalColor.transparent)
        p = QPainter(self.base)
```

```
            self.draw_base(p)
            p.end()
        p = QPainter(self)
        p.drawPixmap(0, 0, self.base)
        p.setRenderHint(QPainter.RenderHint.Antialiasing)
        self.draw_overlay(p)
```

Перевод координат графика в точки экрана — обычная линейная пропорция: $X = X_{\text{лев}} + \dfrac{x - x_0}{x_1 - x_0} W$, $Y = Y_{\text{низ}} - \dfrac{y - y_0}{y_1 - y_0} H$ (ось Y на экране направлена вниз).

7.2. Кривая — картинка, посчитанная numpy

Узкий пик бывает тоньше пикселя, а тысячи отрезков со сглаживанием рисуются медленно. Поэтому кривая строится иначе: для каждого столбца пикселей по формуле известны верх и низ кривой (часть 1, раздел 3.1), и numpy сразу для всех пикселей поля считает, насколько каждый пиксель закрыт линией, заливкой и свечением.

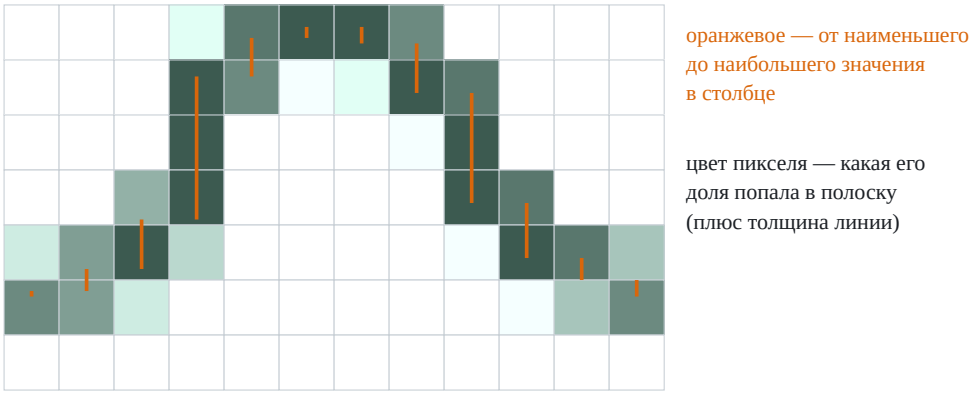


Рис. 4. Пиксели столбца закрашиваются по доле, попавшей в полоску «от низа до верха кривой плюс толщина линии»

Для пикселя в строке y (он занимает отрезок $[y,\ y+1]$) и полоски $[a,\ b]$ доля закрытия

$$\alpha_{\text{лин}} = \min\left(1,\ \max\left(0,\ \min\left(y+1, b\right) - \max\left(y, a\right)\right)\right). \tag{2}$$

Это и есть сглаживание: пиксель на краю полоски закрыт частично и получается полупрозрачным. Три слоя — линия, заливка, свечение — складываются как наложенные полупрозрачные плёнки: $\alpha = 1 - (1 - \alpha_{\text{лин}})(1 - \alpha_{\text{зал}})(1 - \alpha_{\text{свеч}})$.

fabry_perot/widgets.py — Plot.draw_curve

```
def draw_curve(self, p, area, curve):
    ratio = self.devicePixelRatioF()
    width = max(int(round(area.width() * ratio)), 1)
    height = max(int(round(area.height() * ratio)), 1)
    x0, x1 = self.view()
    y0, y1 = self.ylim
    lo, hi = curve.span(np.linspace(x0, x1, width + 1))
    scale = height / (y1 - y0)
    top = ((y1 - hi) * scale).astype(np.float32)[None, :]
    bottom = ((y1 - lo) * scale).astype(np.float32)[None, :]
    rows = np.arange(height, dtype=np.float32)[:, None]
    half = np.float32(0.95 * ratio)
    line = np.minimum(rows + 1, bottom + half)
    line -= np.maximum(rows, top - half)
    np.clip(line, 0, 1, out=line)
```

```
    fill = rows + 1 - top
    np.clip(fill, 0, 1, out=fill)
    fade = np.linspace(0.36 if THEME.dark else 0.24, 0.03, height, dtype=np.float32)[:, None]
    fill *= fade
    clear = 1 - line
    clear *= 1 - fill
    if THEME["glow"]:
        dist = np.maximum(top - rows - 0.5, rows + 0.5 - bottom)
        dist /= np.float32(3.2 * ratio)
        np.clip(dist, 0, 1, out=dist)
        glow = 1 - dist
        glow *= glow
        glow *= np.float32(0.3)
        clear *= 1 - glow
    alpha = ((1 - clear) * 255 + 0.5).astype(np.uint8)
    argb = color_table(curve.color)[alpha]
    image = QImage(argb.tobytes(), width, height, 4 * width,
QImage.Format.Format_ARGB32_Premultiplied)
    image.setDevicePixelRatio(ratio)
    p.drawImage(area.topLeft(), image)
```

Цвет у всей кривой один, различается только прозрачность. Поэтому готовый цвет берётся из таблицы на 256 значений прозрачности: `color_table(curve.color)[alpha]` — одна операция numpy на всю картинку. Таблица хранит цвета в формате «Premultiplied» (цвет заранее умножен на прозрачность) — в нём Qt накладывает картинку без пересчёта.

fabry_perot/widgets.py — color_table

```
def color_table(color):
    c = QColor(color)
    alpha = np.arange(256, dtype=np.uint32)
    r, g, b = ((alpha * k + 127) // 255 for k in (c.red(), c.green(), c.blue()))
    return (alpha << 24) | (r << 16) | (g << 8) | b
```

7.3. Масштаб колёсиком и курсор

fabry_perot/widgets.py — Plot.wheelEvent

```
def wheelEvent(self, event):
    steps = event.angleDelta().y() / 120
    area = self.area()
    if not steps or area.width() < 40:
        return
    a, b = self.zoom or (0.0, 1.0)
    at = min(max((event.position().x() - area.left()) / area.width(), 0.0), 1.0)
    span = min(max((b - a) * 0.8 ** steps, 1e-4), 1.0)
    center = a + at * (b - a)
    a = min(max(center - at * span, 0.0), 1.0 - span)
    self.zoom = None if span > 0.999 else (a, a + span)
    self.refresh()
    event.accept()
```

Видимая часть графика хранится как доли (a, b) всей ширины. Один щелчок колёсика уменьшает ширину в 0,8 раза, причём точка под курсором остаётся на месте: если курсор на доле `at` видимой части, новое начало $a' = c - at \cdot s'$, где c — точка под курсором, s' — новая ширина. Масштаб хранится в долях, поэтому при смене параметров увеличенный участок остаётся на том же месте относительно пиков.

8. КАРТИНА КОЛЕЦ

Картина колец симметрична: цвет пикселя зависит только от расстояния до центра. Поэтому картинка строится в два шага.

1. **Таблица цветов по радиусу** с шагом в четверть пикселя — несколько тысяч значений. Яркость — точное среднее по окну шириной в пиксель (через первообразную, часть 1, раздел 3.2); для двух линий яркости складываются в своих цветах.
2. **Каждому пикселю — цвет из таблицы** по его расстоянию до центра: `table[index]`, где `index` — заранее посчитанный номер ячейки для каждого пикселя (он зависит только от размера картинки и хранится между кадрами).

fabry_perot/widgets.py — RingView.make_image

```
def make_image(self, side, scale=None, center=None):
    lines, half, _, (d, n, f), F, tmax, rho = self.data
    scale = self.scale if scale is None else scale
    cx, cy = self.center if center is None else center
    pixel = 2 * half / (side * scale)
    key = (side, scale, cx, cy)
    index = self.index.get(key)
    if index is None:
        c = (np.arange(side) + 0.5 - side / 2) * pixel
        index = (4 * np.hypot(c[None, :] + cx, c[:, None] + cy) / pixel).astype(np.intp)
        self.index = {key: index}
    count = int(index.max()) + 1
    center_r = (np.arange(count) + 0.5) * pixel / 4
    r1, r2 = np.maximum(center_r - pixel / 2 - rho, 0.0), center_r + pixel / 2 + rho
    s1, s2 = r1 / np.hypot(r1, f), r2 / np.hypot(r2, f)
    if self.glow:
        sigma = max(side / 180, 2.0)
        half_k = int(3 * sigma)
        kernel = np.exp(-0.5 * (np.arange(-half_k, half_k + 1) / sigma) ** 2)
        kernel /= kernel.sum()
    rgb = np.zeros((count, 3))
    for lam, color in lines:
        bright = airy_mean(phase(lam, d, n, s1), phase(lam, d, n, s2), F, tmax) / len(lines)
        if self.glow:
            padded = np.pad(bright, half_k, mode="reflect")
            bright = bright + 0.6 * np.convolve(padded, kernel, mode="valid")
        rgb += np.clip(bright, 0, 1)[:, None] ** (1 / 2.2) * np.array(color)
    alpha = np.clip(half / pixel - (np.arange(count) + 0.5) / 4 + 0.5, 0, 1)
    v = (np.clip(rgb, 0, 1) * alpha[:, None] * 255 + 0.5).astype(np.uint32)
    a = (alpha * 255 + 0.5).astype(np.uint32)
    table = (a << 24) | (v[:, 0] << 16) | (v[:, 1] << 8) | v[:, 2]
    argb = table[index]
    return QImage(argb.tobytes(), side, side, 4 * side,
                  QImage.Format.Format_ARGB32_Premultiplied).copy()
```

Ещё три приёма. **Гамма**: яркость возводится в степень $1 / 2{,}2$ — монитор показывает тёмные оттенки темнее, чем они есть, и без поправки тусклые кольца слились бы с фоном. **Свечение**: к яркости добавляется её размытая копия (свёртка с гауссовым ядром, `np.convolve`), как ореол на фотографии. **Круглый край**: прозрачность плавно растёт на последнем пикселе, край окуляра получается гладким.

Оправа окуляра — ободок и 72 деления — рисуется один раз прямо на картинке (`decorate`), а не при каждом движении мыши.

9. РЕЖИМ ЛАБОРАТОРНОЙ РАБОТЫ И ЖУРНАЛ

Режим включает метод `set_lab`: он убирает с экрана поля λ и A (`set_row_visible`: ни значения, ни ползунка, чтобы его положение ничего не подсказывало), прячет готовые примеры, страницу результатов и кнопку «Копировать», показывает журнал. Фокусное расстояние линзы становится выбором из набора (`ParamRow.use_choices` показывает выпадающий список вместо ползунка), а строка «Расстояние линза — экран L» появляется (`set_row_visible`): кольца резкие, только когда студент поставил экран в фокус. Физика получает значения не из полей, а через `effective`:

fabry_perot/window.py — MainWindow.effective

```
def effective(self):
    v = dict(self.shown)
    if self.lab is not None:
        v["lam"], v["A"] = self.lab.lam, self.lab.A
        if self.unknown_doublet.isChecked():
            v["dlam"] = self.lab.dlam
    return v
```

Всё, что уходит наружу, проверяется на «утечку»: подсказка на кольцах не показывает порядок m (по нему λ вычисляется одним делением), на графике нет автоматических отрезков $\Delta\lambda$ и w, файл CSV содержит только журнал и видимые параметры. Журналу окно отдаёт параметры через `context`, где скрытые — `None`.

Журнал (`journal.py`) хранит серии точек по заданиям. Правый щелчок по графику посылает сигнал `picked(x, y)`; журнал решает, что с ним делать, по типу серии: для «пары» запоминает первую точку и по второй записывает разность, для колец — номер кольца и r^2:

fabry_perot/journal.py — Journal.plot_pick

```
def plot_pick(self, x, unit, y):
    s = self.series()
    x_pm = x * 1000.0 if unit == "нм" else x
    if s.kind == "pair":
        if self.pending is None:
            self.pending = x_pm
            self.notify(f"Первая точка: {num(x_pm, 5)} пм. Щёлкните правой кнопкой по второй
точке.")
            self.hint.setText(f"Первая точка: x₁ = {num(x_pm, 5)} пм — щёлкните вторую.")
            return
        first, self.pending = self.pending, None
        ctx = self.context()
        self.add(s.xfunc(ctx[s.xkey]), abs(x_pm - first),
                 f"x₁ = {num(first, 5)}, x₂ = {num(x_pm, 5)} пм")
        self.series_changed()
    elif s.key == "tmax":
        self.add(self.context()["R"], y, f"λ − λ₁ = {num(x_pm, 5)} пм")
    elif s.key == "free":
        self.add(x_pm, y, "график T(λ): x — отступ, пм")
    else:
        self.notify("Эта серия записывается " + ("щелчком по кольцам." if s.kind == "ring"
                                                 else "кнопкой «Записать»."))
```

После каждой записи журнал заново проводит прямую (`lab.fit_line`), рисует точки и прямую тем же классом `Plot`, что и основные графики, и сохраняет данные в настройки — работу можно продолжить после перезапуска.

9.1. Быстрый шаг стрелкой

Чтобы стрелки ↑ ↓ давали короткий переход (0,15 с), а набранное число — плавный (0,85 с), поле ввода — подкласс, который помечает изменения «шагом»:

fabry_perot/params.py — StepSpin

```
class StepSpin(QtWidgets.QDoubleSpinBox):
    stepping = False

    def stepBy(self, steps):
        self.stepping = True
        super().stepBy(steps)
        self.stepping = False
```

9.2. Лупа на картине колец

fabry_perot/widgets.py — RingView.wheelEvent

```
def wheelEvent(self, event):
    if self.data is None:
        return
    steps = event.angleDelta().y() / 120
    new = min(max(self.scale * 1.25 ** steps, 1.0), 40.0)
    if new == self.scale:
        return
    x, y = self.to_screen(event.position())
    k = self.scale / new
    self.center = (x - (x - self.center[0]) * k, y - (y - self.center[1]) * k)
    self.scale = new
    if new == 1.0:
        self.center = (0.0, 0.0)
    self.image = None
    self.update()
    event.accept()
```

Вид хранит увеличение и точку экрана прибора в центре вида. При повороте колёсика точка под курсором остаётся на месте: новый центр $c' = p - (p - c)\,s / s'$, где p — точка под курсором, s и s' — старое и новое увеличение. Картинка по-прежнему строится по таблице цветов: для каждого пикселя вида считается расстояние до центра колец.

10. СОХРАНЕНИЕ И НАСТРОЙКИ

fabry_perot/window.py — MainWindow.write_csv

```
def write_csv(self, path):
    if not path.lower().endswith(".csv"):
        path += ".csv"
    try:
        with open(path, "w", newline="", encoding="utf-8-sig") as file:
            out = csv.writer(file, delimiter=";")
            if self.lab is not None:
                self.write_lab_csv(out)
                return self.statusBar().showMessage(f"Журнал сохранён: {path}", 6000)
            out.writerow(["Параметр", "Значение", "Единица"])
            for row in self.params.values():
                out.writerow([row.name, plain(self.shown[row.key]), row.unit])
            out.writerow(["Вторая линия", "включена" if self.second.isChecked() else "выключена"])
            out.writerow([])
            out.writerow(["Величина", "По графику", "По формуле", "Единица"])
            for name, got, expect, unit, _ in self.results:
                out.writerow([name, plain(got), plain(expect), unit])
            out.writerow([])
            out.writerow(["Кольцо №", "r по картинке, мм", "r по формуле, мм"])
```

```
            measured, predicted = self.radii
            for k in range(max(len(measured), len(predicted))):
                got = measured[k] * 1e3 if k < len(measured) else None
                expect = predicted[k] * 1e3 if k < len(predicted) else None
                out.writerow([k + 1, plain(got), plain(expect)])
            out.writerow([])
            out.writerow(["λ, нм", "Пропускание T"])
            x, T = self.curve
            step = max(1, len(x) // 20000)
            for a, b in zip(x[::step], T[::step]):
                out.writerow([f"{a * 1e9:.6f}".replace(".", ","), plain(b)])
            journal = self.journal.csv_rows()
            if journal:
                out.writerow([])
                out.writerow(["Журнал измерений"])
                for row in journal:
                    out.writerow(row)
    except OSError as error:
        QtWidgets.QMessageBox.warning(self, "Ошибка", f"Не удалось сохранить файл:\n{error}")
        return
    self.statusBar().showMessage(f"Таблица сохранена: {path}", 6000)
```

Таблица сохраняется в кодировке `utf-8-sig` с разделителем «;» и десятичной запятой — так русский Excel открывает файл двойным щелчком без «кракозябр». Параметры, тема и размер окна запоминаются между запусками в INI-файле (`QSettings`): на Windows — в папке `%APPDATA%`, на macOS и Linux — в домашней папке пользователя.

11. СКОРОСТЬ НА СЛАБЫХ КОМПЬЮТЕРАХ

Программа рассчитана на компьютеры в аудиториях, например на Core i5 второго поколения со встроенной графикой. Видеокарта не нужна: всё рисует процессор. Время одного кадра (пересчёт и перерисовка всех графиков) на одном ядре:

Таблица 1

Время кадра, мс

Режим	Первая версия	Вторая версия
He-Ne лазер, $R = 0,9$	31	10
$R = 0,99$, $d = 20$ мм	129	11
Экран 100 мм, $f = 50$ мм (22 тыс. колец)	167	11
Две линии далеко друг от друга	354	11

Принципы, за счёт которых это получено:

- считать по формуле то, что можно посчитать по формуле (наибольшее и наименьшее значение в столбце, среднее по пикселю), вместо перебора миллионов точек;
- все вычисления по пикселям — действиями numpy над целыми массивами, без циклов Python;
- не рисовать повторно то, что не изменилось: рисунок-«кэш» графика, оправа окуляра, номера ячеек картинки колец;
- склеивать частые события (таймер 15 мс) и вести анимацию по часам, а не по кадрам.

12. ЗАПУСК И СБОРКА ПРОГРАММЫ

fabry_perot/app.py — create_app

```
def create_app(argv=None):
    if not QT6:
        QtWidgets.QApplication.setAttribute(Qt.ApplicationAttribute.AA_EnableHighDpiScaling)
        QtWidgets.QApplication.setAttribute(Qt.ApplicationAttribute.AA_UseHighDpiPixmaps)
    try:
        QtGui.QGuiApplication.setHighDpiScaleFactorRoundingPolicy(
            Qt.HighDpiScaleFactorRoundingPolicy.PassThrough)
    except AttributeError:
        pass
    app = QtWidgets.QApplication(sys.argv if argv is None else argv)
    app.setApplicationName("FabryPerot")
    app.setStyle("Fusion")
    font = QFont(app.font())
    font.setFamilies(FONTS)
    app.setFont(font)
    app.setWindowIcon(app_icon())
    QtCore.QLocale.setDefault(QtCore.QLocale(QtCore.QLocale.Language.Russian,
QtCore.QLocale.Country.Russia))
    return app
```

Стиль `Fusion` одинаково выглядит на всех системах. Для Qt 5 чёткость на экранах с масштабом 125–150 % включается вручную (`AA_EnableHighDpiScaling`). Запятая в полях ввода — из русской локали. Ключ `--selftest` открывает окно, делает расчёт и закрывается — так проверяется собранная программа.

Сборка. Программа превращается в запускаемый файл утилитой **PyInstaller**: она упаковывает Python, библиотеки и код в один файл. Сборку выполняет сервис GitHub Actions при каждом изменении кода (файл `.github/workflows/build.yml`):

Таблица 2

Сборки

Система	Состав	Результат
Windows 7, 8, 8.1, 10, 11	Python 3.8 (32 бит), PyQt5 5.15 — последние версии, работающие на Windows 7; внутрь кладётся библиотека UCRT	один `.exe` и portable-папка в `.zip`
macOS 11 и новее	Python 3.12, PyQt6 6.7; отдельно для Apple Silicon и Intel	`.app` в образе `.dmg`
Linux x86-64	Python 3.11, PyQt6 6.7, собирается на Ubuntu 22.04	файл и `install.sh` в `.tar.gz`

32-битная сборка для Windows запускается и на 32-, и на 64-битных системах. Перед выкладкой каждая сборка проходит тесты и пробный запуск с ключом `--selftest`.

13. ВОПРОСЫ НА ЗАЩИТЕ

На чём написан интерфейс и почему не matplotlib?

На Qt (PyQt6, на Windows 7 — PyQt5). Графики нарисованы своим классом на QPainter и numpy: это быстрее, позволяет точно рисовать узкие пики и не требует лишних библиотек.

Как работает плавный переход?

Программа хранит цель (значение в поле) и то, что сейчас на картинке. Таймер 60 раз в секунду сдвигает картинку к цели по S-образной функции плавности; время перехода отсчитывается по часам, поэтому на медленном компьютере переход длится столько же, просто с меньшим числом кадров.

Почему ползунок меняет картину сразу, а поле — плавно?

Строка параметра знает источник изменения (флажок `from_slider`). При движении ползунка пользователь сам «ведёт» картину, плавность только мешала бы; при вводе числа видно, как картина переходит от старого состояния к новому.

Как сделана тёмная тема?

Все цвета — в словарях двух палитр. Виджеты берут цвета из общего объекта темы при рисовании, стандартные элементы оформлены таблицей стилей Qt, собранной из текущей палитры. Смена темы — новая палитра, новая таблица стилей и перерисовка.

Почему программа быстрая на старом компьютере?

Картинки считаются по формулам для каждого пикселя, а не по миллионам точек; все операции — над массивами numpy; неизменные части рисунка хранятся готовыми. Кадр занимает около 10 мс при любых параметрах.

Как программа работает на Windows 7?

Для Windows собирается версия на Python 3.8 и Qt 5.15 — последних, поддерживающих Windows 7. Различия Qt 5 и Qt 6 собраны в модуле `qt.py`, остальной код общий.